Digital Signal Processing Assignment

Task 1- Time and frequency domain analysis

```
%% Task 1, Analysis

clc;
clear all;
close all;

%Reads the audio file.
[x, fs] = audioread("C:\Users\mail2\OneDrive\Documents\MATLAB\Matlab Assignment\Daniel Hawkins.wav"); %Reads two variables, x signal and sampling frequency.
fprintf('Sampling frequency: %.0f Hz\n', fs);

%Time domain analysis.
ts = 1/fs;
tmax = length(x)/fs; %Length(x) is the total number of samples in the audio signal. Total samples / samples per second = tmax.
fprintf('Signal length: %.2f seconds\n', tmax); %Prints characteristics of the audio file.
fprintf('Total samples: %.0f\n', length(x));
t = 0:ts:tmax-ts; %Time vector
figure; plot(t, x);xlabel('Time(s)');ylabel('Amplitude'); title('Input audio in time domain'); grid on; %Time domain plot of the audio file.

%Frequency domain analysis.
figure; pspectrum(x,fs); %Power spectrum of the input audio (frequency domain).

%Frequency domain analysis using fft.
n = length(x); %Total number of samples.
x_fft = fft(x)/n;%FFT computation, Normalisation for the bins.
x_mag = abs(x_fft); %Takes absolute values of FFT to show amplitude of frequency components.
x_mag = x_mag(1:floor(n/2)+1); %Only positive frequencies needed as fft gives mirror image.
x_mag(2:end-1) = 2 * x_mag(2:end-1); %Scale amplitude as negative frequencies took half the amplitude which we are discarding.
x_dB = 20 * log10(x_mag);%Logarithmic scaling.

% Frequency vector
f = (0:floor(n/2)) * fs / n; %Frequency = bin indicies * frequency resolution (fs/n). FFT only gives bin indicies not frequencies.

%Plot frequency domain.
figure; plot(f/1000, x_mag); xlabel('Frequency (kHz)'); ylabel('Amplitude'); title('Frequency Domain (FFT) of input audio signal'); grid on;
figure; plot(f/1000, x_dB);  xlabel('Frequency (kHz)'); ylabel('Magnitude (dB)'); title('Logarithmic scaling of FFT'); grid on; %We can now see the weak signals aswell.

```

Figure 1: Waveform analysis

Figure 1 shows the audioread MATLAB function reading the audio file as a matrix of x (the signal) and fs which is the sampling frequency. The sampling time is defined by 1/fs. The sampling frequency, signal length and total samples are printed to the console (note that samples have a .0 floating point since we can't get decimal samples). The time vector is created as we are plotting signal against time intervals of sampling time up to the maximum time (tmax – ts as to not get an extra sample).

Sampling frequency = 96000 Hz, Signal length = 2.47 seconds, Total samples: 236736.

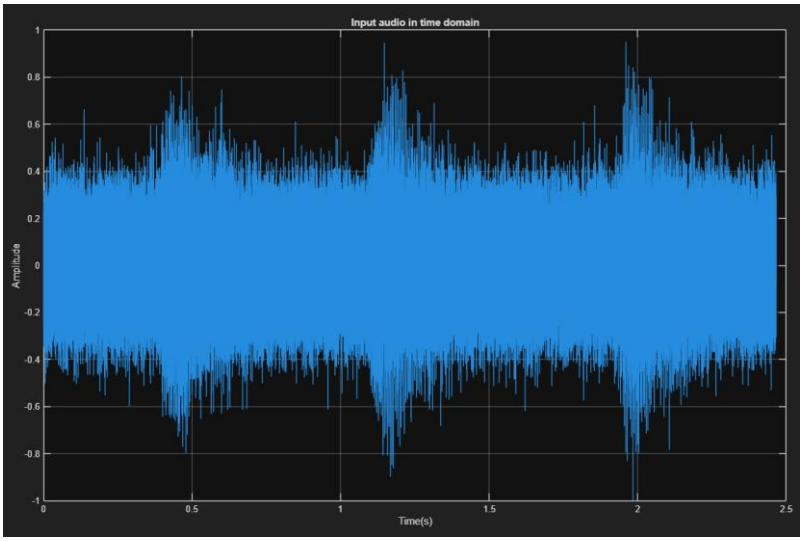

Figure 2: Input audio in the time domain

Figure 2 shows the time domain waveform. There is some structure to this waveform with clear amplitudes of the message however is very dominated by noise as the signal fluctuates frequently due to the carrier frequency and possibly other noise factors.

Figure 3 shows the power spectrum function and is there just for comparison for the FFT computation to make sure that it the FFT was done correctly.

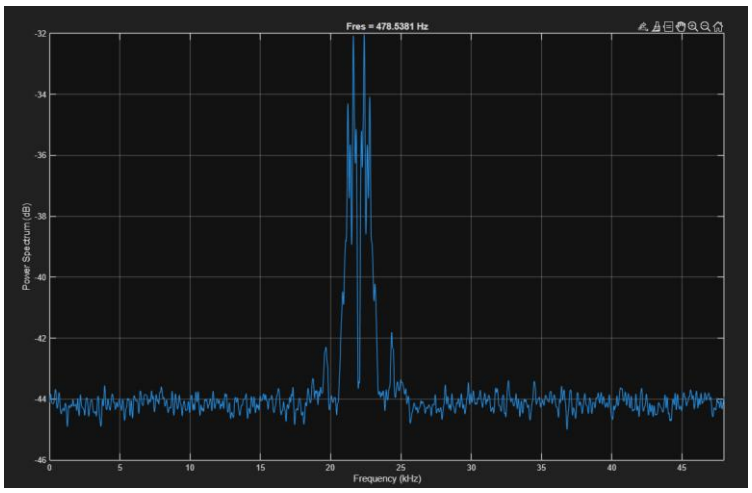

Figure 3: Power spectrum of input audio

The normalisation is done to make sure the FFT amplitude represents the signal correctly. Otherwise, the amplitudes would scale with the number of samples. The positive values are taken, and the negative values are discarded as the FFT gives a mirrored version of itself. The abs function is so that we only take real numbers as the FFT outputs complex numbers.

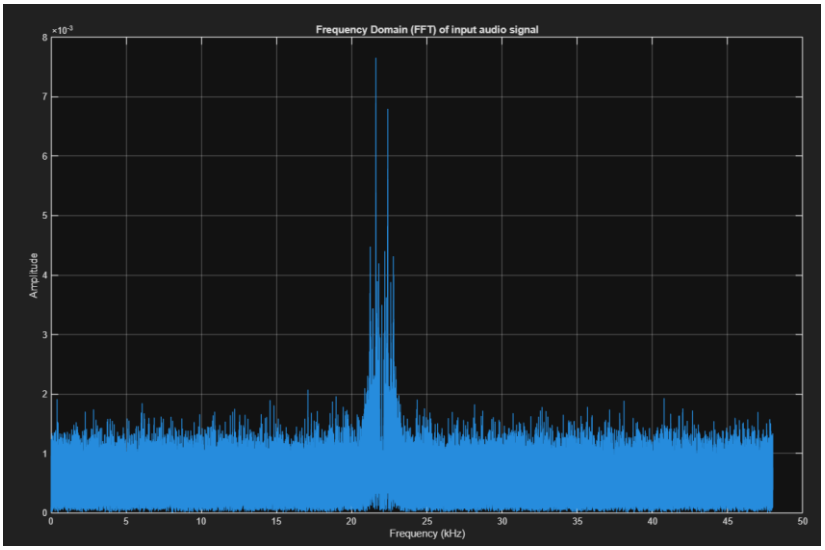

Figure 4: FFT of the input signal

Figure 4 shows the output and figure 5 is logarithmically scaled so that the weak and strong signals can be seen in the same graph as it is easier to see differences using this method. Figure 5 shows a clear peak for the carrier frequency and its sidebands. There

are clear weak and strong signals in the frequency domain. Comparing the power spectrum to this it can be said that the carrier frequency is around 22kHz however later tasks will confirm this. There is a lot of noise around the carrier frequency + sidebands that we will want to filter out to get a clearer message.

Sidebands occur at carrier frequency ± message frequency. Therefore, the spectrum of this signal was determined to have an 8kHz bandwidth with fmin = 18kHz and fmax = 26kHz. This was determined just from inspection of the spectrum in figure 5.

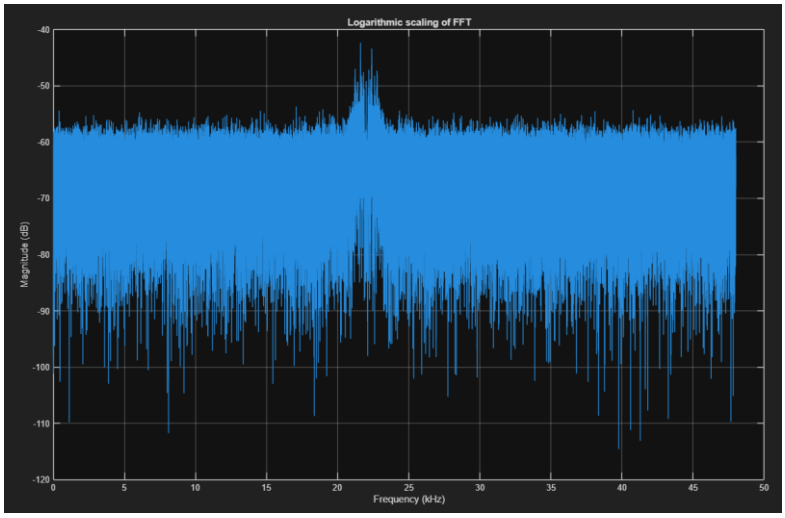

Figure 5: Logarithmic scaling of the FFT

Windows are now applied and compared in both the time and frequency domains. Figure 7 and 8 shows the differences between the different windows in the respective domain. Applying windows should reduce the spectral leakage that occurs in the original signal.

```
%Window functions.
bw = blackman(n);
hanw = hanning(n);
hamw = hamming(n);

% Apply the Hamming window to the signal
x_hamw = x .* hamw; % Element-wise multiplication of the signal with the Hamming window
x_hamw_fft = fft(x_hamw)/sum(hamw); %FFT of the Hamming windowed signal normalised.
x_hamw_amp = abs(x_hamw_fft); % Amplitude of the Hamming windowed FFT
x_hamw_amp = x_hamw_amp(1:floor(n/2)+1); % Only positive frequencies
x_hamw_amp(2:end-1) = 2 * x_hamw_amp(2:end-1);
x_hamw_dB = 20 * log10(x_hamw_amp);


% Apply the Hanning window to the signal
x_hanw = x .* hanw; % Element-wise multiplication of the signal with the Hanning window
x_hanw_fft = fft(x_hanw)/sum(hanw); %FFT of the Hanning windowed signal normalised
x_hanw_amp = abs(x_hanw_fft); % Amplitude of the Hanning windowed FFT
x_hanw_amp = x_hanw_amp(1:floor(n/2)+1); % Only positive frequencies
x_hanw_amp(2:end-1) = 2 * x_hanw_amp(2:end-1);
x_hanw_dB = 20 * log10(x_hanw_amp);

%Apply the blackman window to the signal.
x_bw = x .* bw; % Element-wise multiplication of the signal with the window
x_bw_fft = fft(x_bw)/sum(bw); %FFT of the windowed signal normalised.
x_bw_amp = abs(x_bw_fft); % Amplitude of the windowed FFT
x_bw_amp = x_bw_amp(1:floor(n/2)+1); % Only positive frequencies
x_bw_amp(2:end-1) = 2 * x_bw_amp(2:end-1);
x_bw_dB = 20 * log10(x_bw_amp);

%FFT window comparison.
figure;
hold on;
plot(f/1000, x_dB, 'r', 'DisplayName','Input audio signal');
plot(f/1000, x_hamw_dB, 'g', 'DisplayName', 'Hamming Window');
plot(f/1000, x_hanw_dB, 'y', 'DisplayName', 'Hanning Window');
plot(f/1000, x_bw_dB, 'b', 'DisplayName', 'Blackman Window');
hold off;
xlabel('Frequency(kHz)');
ylabel('Amplitude');
title('Window Functions frequency Comparison');
legend('show');
grid on;
figure;
hold on;
plot(t, x, 'r', 'DisplayName','Input audio signal');
plot(t, x_hamw, 'g', 'DisplayName', 'Hamming Window');
plot(t, x_hanw, 'y', 'DisplayName', 'Hanning Window');
plot(t, x_bw, 'b', 'DisplayName', 'Blackman Window');
hold off;
xlabel('Time(s)');
ylabel('Amplitude');
title('Window Functions time domain comparison');
legend('show');
grid on;


```

Figure 6: Window code

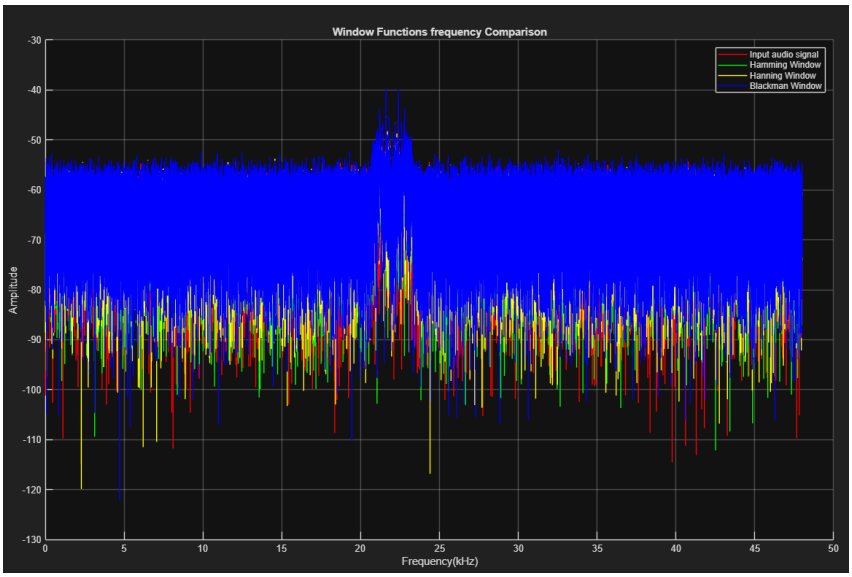

Figure 7: Windows applied in the frequency domain

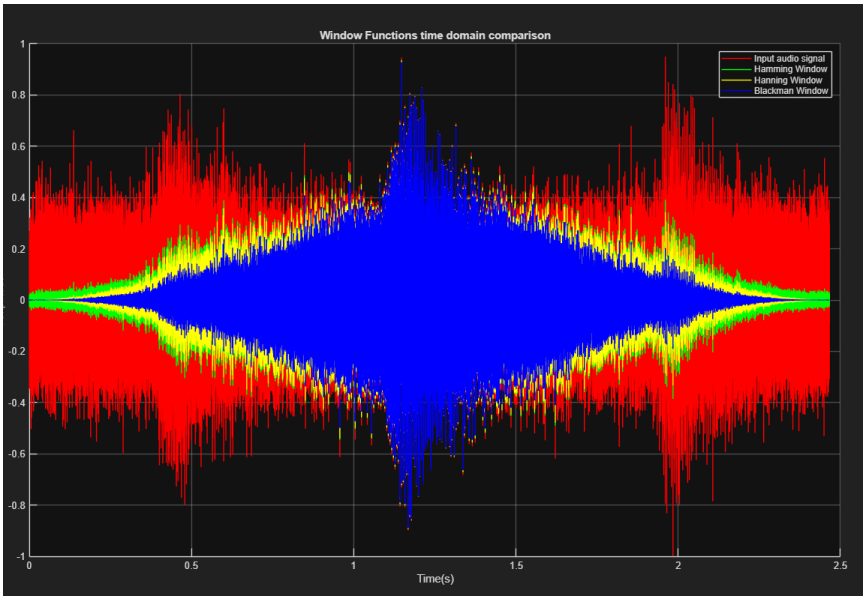

Figure 8: Windows applied in the time domain

Figure 8 makes it clear that each window has different tapering and roll-off.

As the frequencies falls between two bins, spectral leakage can occur, shown in figure 9 when zooming in on the peaks of figure 7 (The peaks are wide and spread out). The frequency resolution is fs/n which is 96000/236736 = 0.4. This means the FFT can distinguish frequencies 0.4 Hz apart from each other.

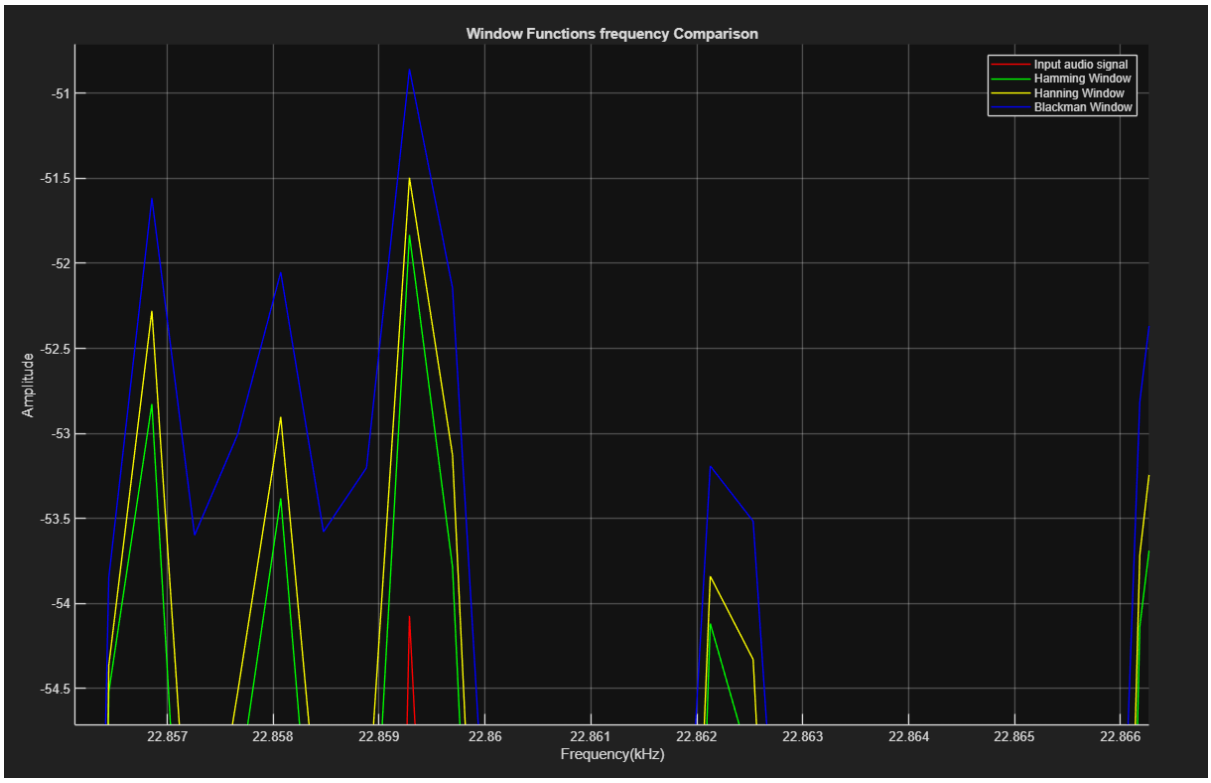

Figure 9: Spectral leakage

Task 2- Bandpass filter

Fmin and Fmax were determined to be 18kHz and 26kHz respectively.

```
%% Task 2 Pass band filter.

t = 0:ts:(length(x)-1)*ts;
fmin = 18000;
fmax = 26000;
fst1 = 16000; %Stopband frequencies.
fst2 = 28000;
f1 = fmin/fs; %Normalised cut-off frequencies.
f2 = fmax/fs;
m = 301; %Number of orders.
N =2*m+1; %Total tap number
w1 = 2 * pi * f1; %Omega
w2 = 2 * pi * f2;

%Bandpass Filter.
for n = 1:m %Only calculates RHS of impulse response as LHS are negative order taps.
    h(n) = 2*f2*sin(n*w2)/(n*w2) - 2*f1*sin(n*w1)/(n*w1); %Ideal frequency response.
end

h = [fliplr(h) (2*f2)-(2*f1) h]; %Impulse response of ideal filter response in here.

%Apply windows
bw = blackman(N)';
h_windowed = bw .* h;
Filter_output= conv(x, h_windowed, 'same'); % 'same' makes sure the array lengths are the same.
% Filtered output with original signal.
figure; plot(t, Filter_output); title('Time domain bandpass filtered output');xlabel('Times(s)');ylabel('Amplitude');grid on; %Actual output.
figure; freqz(h_windowed, 1, 8192, fs); title('Frequency Response of the bandpass filter');grid on; %8192 is arbitrary and is the number of frequency points used for calculation.
% freqz is used to show the frequency response of the filter whilst pspectrum used for frequency content of the output signal.
xlim([0 48]); %Digital signals can only be represented in the nyquist frequency accurately.

```

Figure 10: Bandpass code

Here, fmin and fmax are defined and stopband frequencies are also defined as ±2kHz of fmin and fmax. The cutoff frequencies are then normalised so that angular frequency will be between 0 and pi (angular frequency = 2*π *normalised frequency as defined in figure 10). This is important so that the impulse response of the ideal filter formula works as seen in figure 11.

Filter type	$h_D[n],\ n \neq 0$	$h_D[n],\ n = 0$
Low-pass	$2F_c \dfrac{\sin(n\Omega_c)}{n\Omega_c}$	$2F_c$
High-pass	$-2F_c \dfrac{\sin(n\Omega_c)}{n\Omega_c}$	$1-2F_c$
Band-pass	$2F_2 \dfrac{\sin(n\Omega_2)}{n\Omega_2} - 2F_1 \dfrac{\sin(n\Omega_1)}{n\Omega_1}$	$2F_2 - 2F_1$
Band-stop	$1-\left[2F_2 \dfrac{\sin(n\Omega_2)}{n\Omega_2} - 2F_1 \dfrac{\sin(n\Omega_1)}{n\Omega_1}\right]$	$1-[2F_2 - 2F_1]$

Figure 11: Impulse response for an ideal filter

The band-pass formula is applied using a for loop between samples 1:m as only positive taps can be calculated because MATLAB cannot have negative array indices. FIR impulse response is symmetric so using the fliplr function, the left-hand side can also be shown. Now the full impulse response of an ideal filter has been constructed.

The Blackman window was chosen because the original wav file was extremely noisy and figure 8 shows that it has the strongest tapering the fastest roll-off.

Now using Gibbs phenomenon (multiplication in time domain is equivalent to convolution in frequency domain), convolution is performed between the Blackman window and the impulse response h.

Comparing figure 12 to figure 2, the noise has already been reduced greatly, and the modulated signals are clearer and more visible.

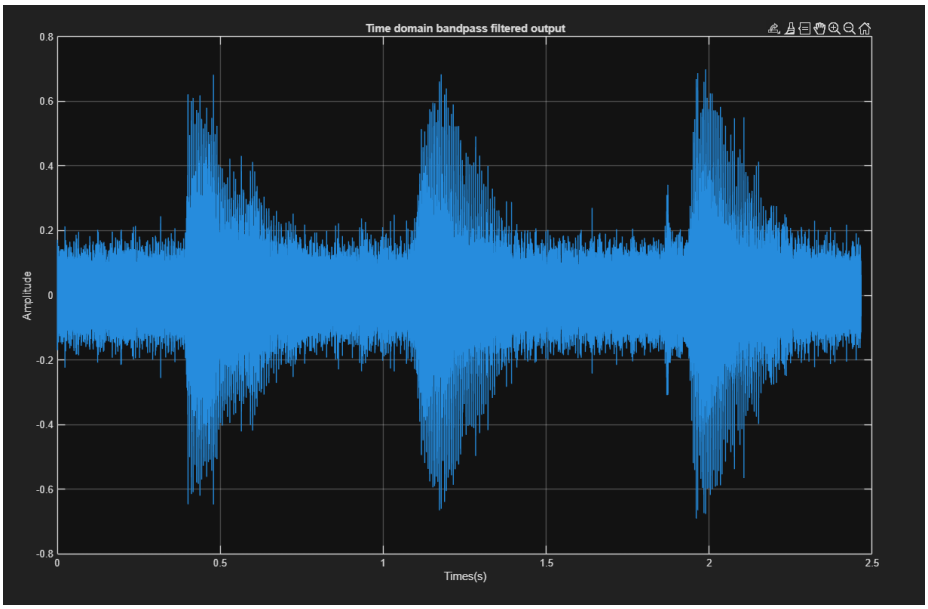

Figure 12: Time domain of band-passed signal

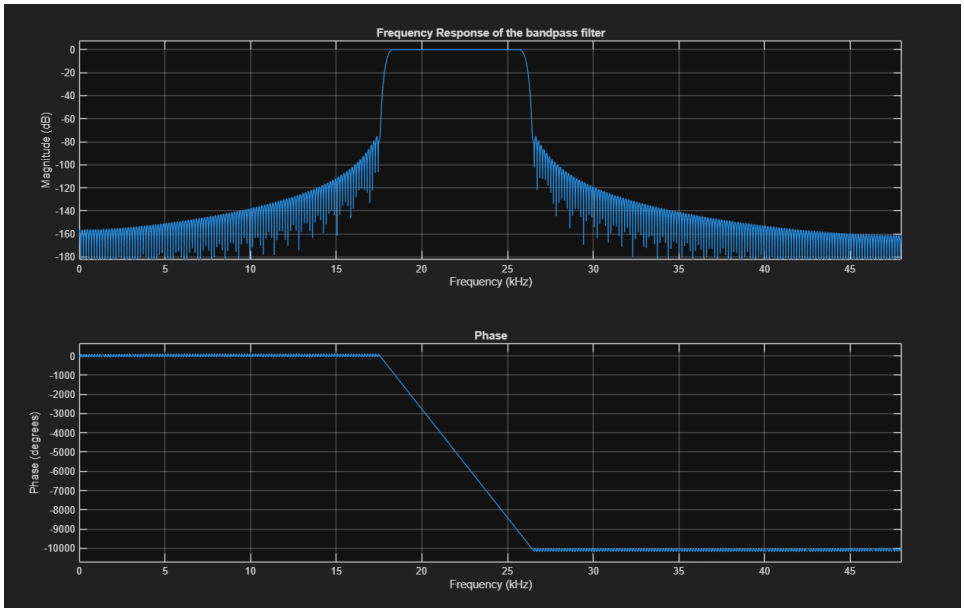

Figure 13: Frequency response of FIR bandpass filter

The passband of the filter is very flat and meets the criteria of the 0.1dB max ripple. After cutoff frequency, the power starts to attenuate and after the stopband frequencies, the power has already been attenuated by 80dB.

Task 3- Carrier recovery and mixing

```
%% Task3, Square law.
fsq = Filter_output .^2; %Filter output squared.
fsq_fft = fft(fsq);
fsq_fft_mag = abs(fsq_fft);
frequencies = (0:length(fsq_fft)-1) * fs  / length(fsq_fft);
figure;plot(frequencies/1000,(fsq_fft_mag + eps)); xlabel('Frequency (kHz)');ylabel('Amplitude');title('FFT of bandpass filter squared'); grid on; %/1000 for kHz eps to avoid log(0) becoming -infinity.
xlim([0 48]); %No need to remove mirror image as it is just for analysis to see the 2*fc component so can just hide outside the nyquist frequency.
fcut = 22000; %Carrier frequency.
t_filter = (0:length(Filter_output)-1) * ts; %Used to match the time of the filter output to the local_carrier.
local_carrier = cos((2 * pi * fcut * t_filter)+ (0));
local_carrier = local_carrier(:); %converts row to column vector to match filter_output.
carrier_recovery = local_carrier .* Filter_output;

figure; plot(t_filter, carrier_recovery);title('Carrier recovery output in time domain');xlabel('Time(s)'); ylabel('Amplitude');grid on; %Carrier recovery in time domain.

carrier_recovery_fft = fft(carrier_recovery);
carrier_recovery_mag = abs(carrier_recovery_fft);
frequencies_cr = (0:length(carrier_recovery_fft)-1) * fs / length(carrier_recovery_fft);

figure;
plot(frequencies_cr/1000, carrier_recovery_mag); %/1000 for kHz.
xlim([0 48]);
xlabel('Frequency (kHz)'); ylabel('Amplitude'); title('Carrier recovery FFT output in frequency domain'); grid on;

```

Figure 14: Square Law code

The filter output is squared and then an FFT of this is computed.

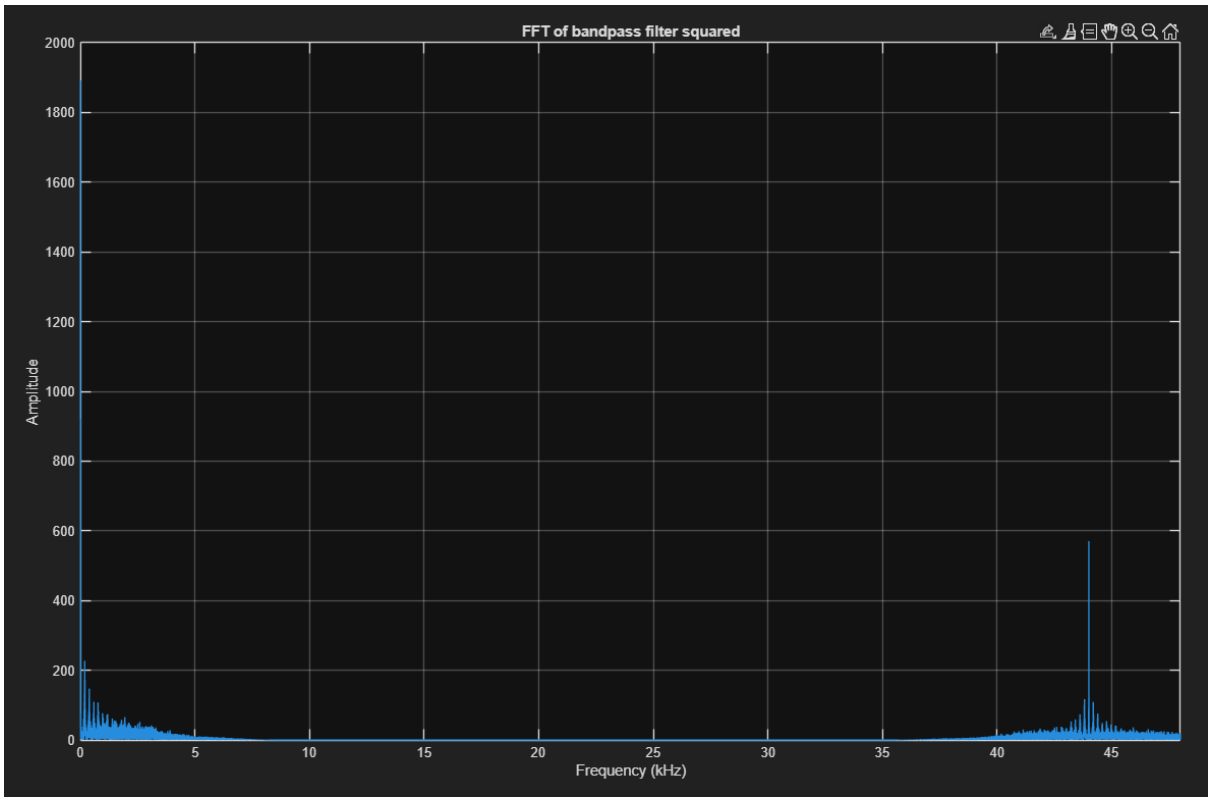

Figure 15: FFT of squared filter output

The x axis has been limited to the Nyquist frequency as only values in this range can be accurately represented digitally. There is a clear peak at 44kHz which represents 2* carrier frequency therefore this graph confirms that the carrier frequency was indeed 22kHz. The message can now also be seen at the low frequency spectrum as a lot of noise has now been removed.

Now that the carrier frequency is confirmed, the local carrier signal (cos ωct + ϕ) can be used to multiply with the bandpass filter output to get figure 16 and 17.

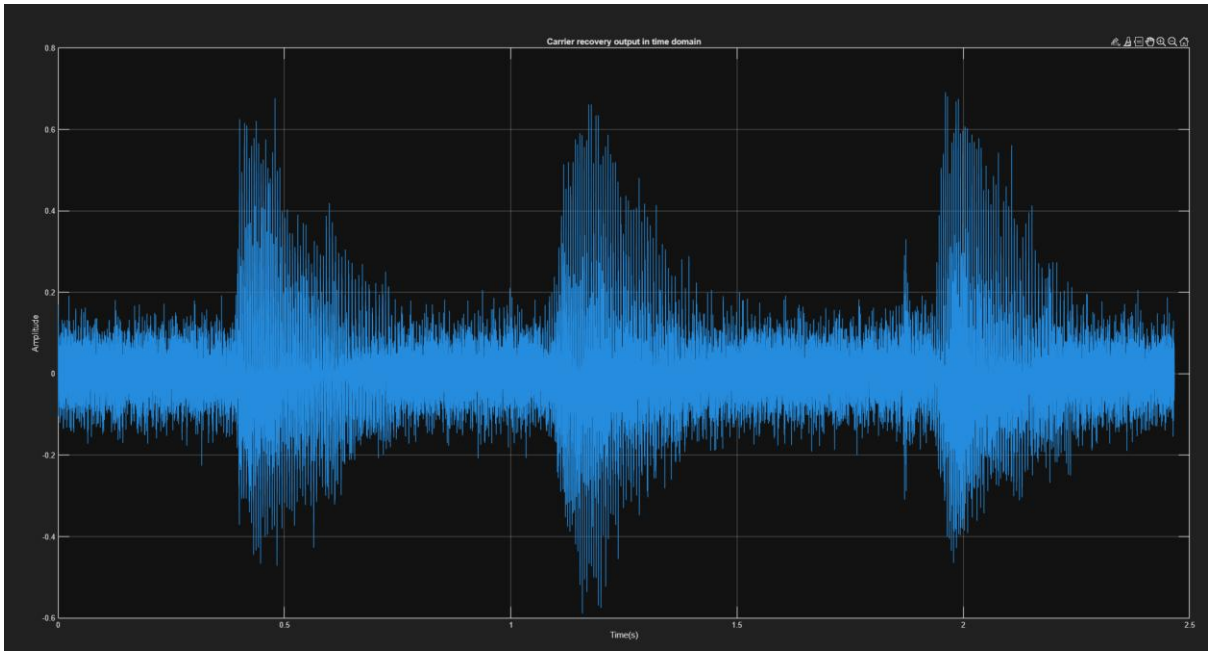

Figure 16: Time domain of carrier recovery

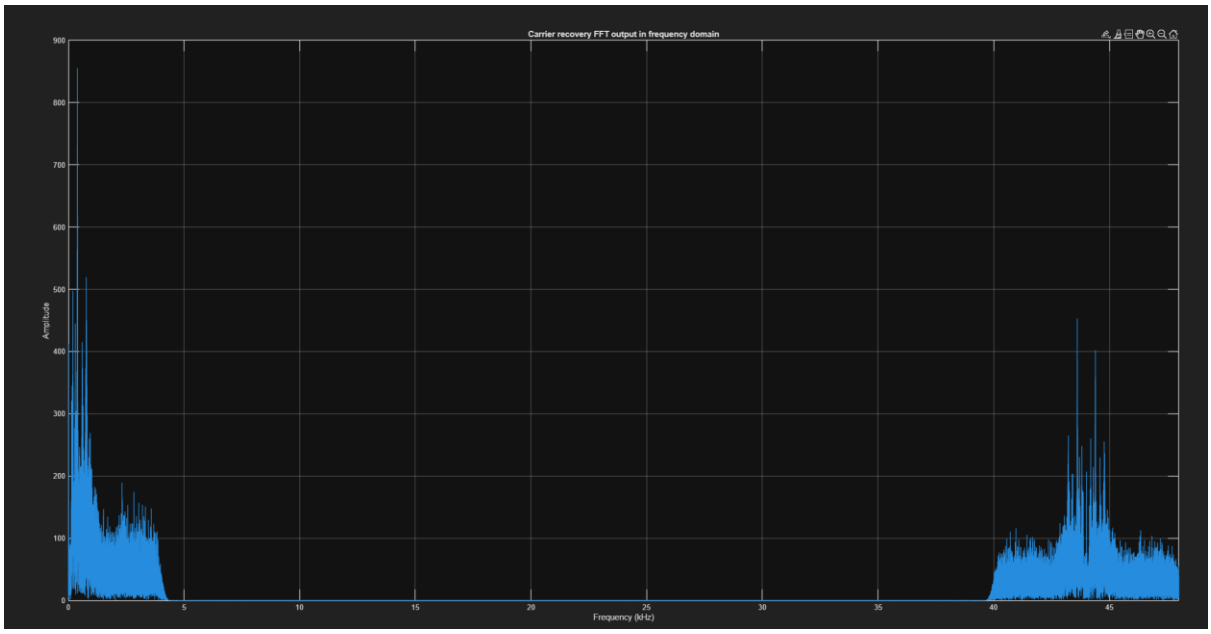

Figure 17: Frequency domain of carrier recovery

Figure 16 shows clear modulated envelopes showing the message is in three bursts. There is still noise however the amplitude differences to the envelopes should mean that it can be listenable however figure 17 shows that we must first apply a low pass

filter to remove the carrier frequency*2 before listening. The carrier recovery has reduced spectral leakage as seen in figure 18 and 19. It is clear to see

that the peaks are much more narrow after the carrier recovery was applied.

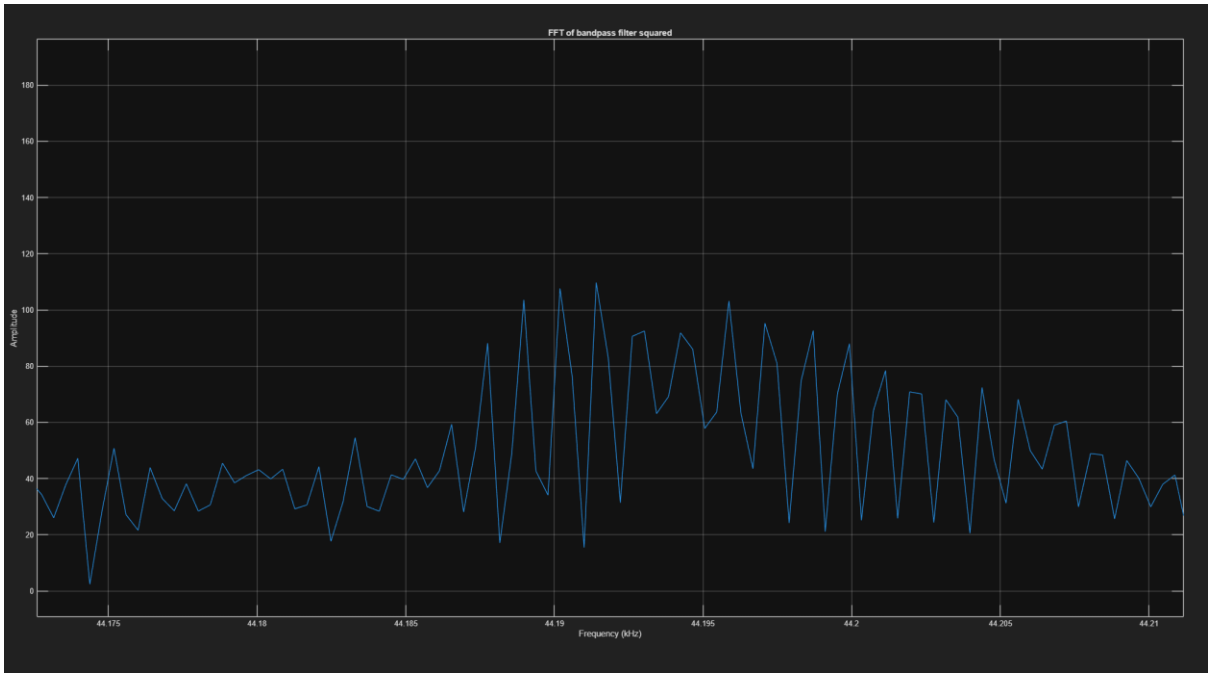

Figure 18: Spectral leakage bandpass filter

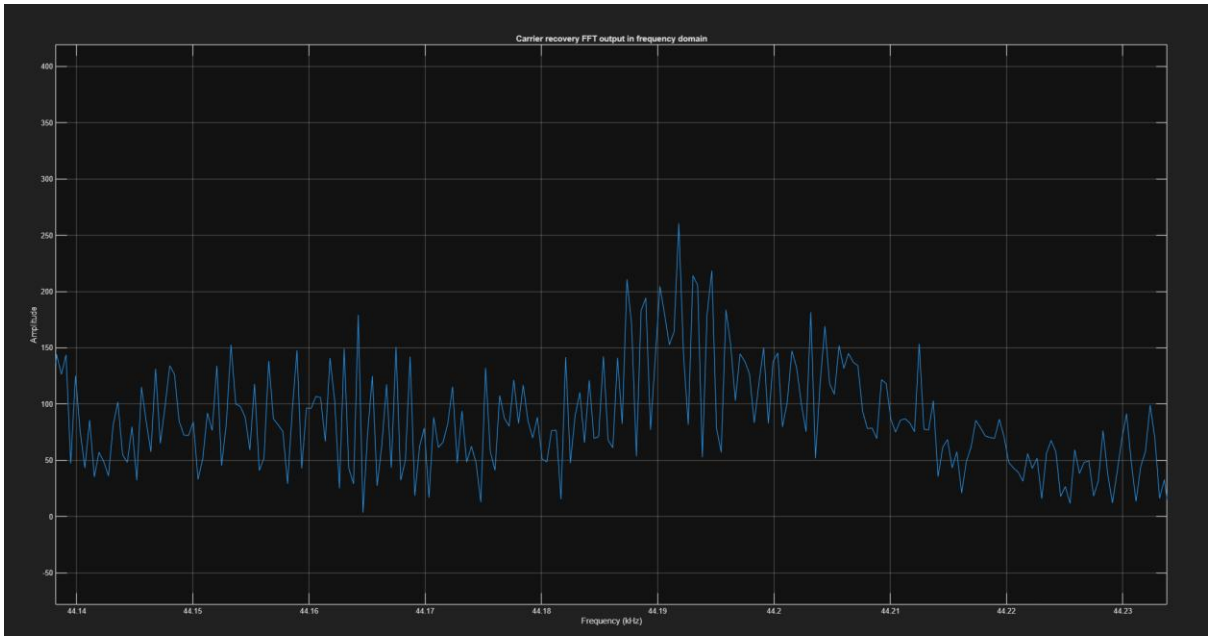

Figure 19: Spectral leakage carrier recovery

$$H(z) = \frac{b(z)}{a(z)} = \frac{b_0 + b_1 z^{-1} + \cdots + b_N z^{-N}}{1 + a_1 z^{-1} + \cdots + a_N z^{-N}}$$

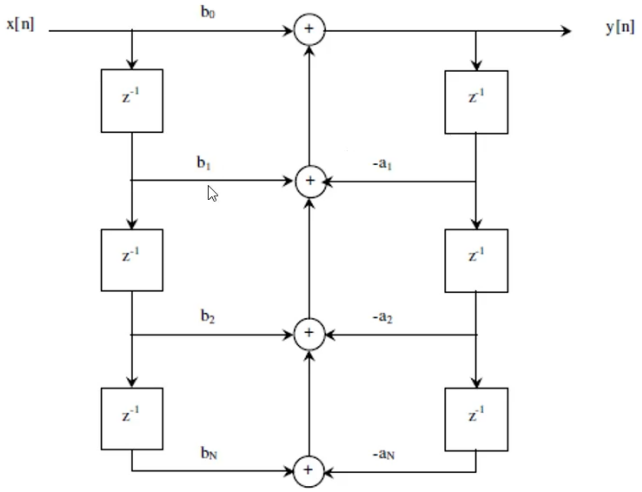

Figure 20: IIR filter realisation

```
%% Task 4, Lowpass filter.
fcut = 4000; %Cutoff frequency.
order = 4;
[b, a] = butter(order, fcut/(fs/2)); %fs/2 for normalisation.
figure;
freqz(b, a, 8192, fs); %Verifying frequency response.
title('IIR Filter Frequency Response');
grid on;

iir_lowpass = zeros(size(carrier_recovery));
ff = zeros(order+1,1);%Feedforward line.
fb = zeros(order,1); %feedback line.

for i=1:length(carrier_recovery) %This loop is the shift registers to apply the feedback and feedforward. Process each sample in the input signal.
 for n=order+1:-1:2 %For every feedforward element. Only goes to order 2 as register 1 takes the input (not shifted from a tap).
        ff(n) = ff(n-1); %Shift input delay line by one. (Last sample is forgotten).
    end
    ff(1) = carrier_recovery(i); %New sample of input is inputted into register 1.
    Count = 0;
    for n=1:order+1 %Apply forward coefficients (FIR).
        Count = Count + b(n)*ff(n); %Add the weights * past input + count so it adds every tap and also the input.
    end
    for n=2:order+1 %Apply feedback coefficients, starts at 2 because we cannout use current output.
        Count = Count - a(n)*fb(n-1); %The count minuses the feedback coefficients * past outputs.
    end
     Count = Count/a(1); %Normalisation.
    iir_lowpass(i) = Count; %Output value.
    for n=order:-1:2 %Shift output delay line down 1 sample (last sample is forgotten).
        fb(n) = fb(n-1);
    end
    fb(1) = Count; %Store latest output in delay line.
end
figure; plot(t, iir_lowpass); title('IIR lpf output in time domain'); xlabel('Time(s)'); ylabel('Amplitude'); grid on;
fft_iir = fft(iir_lowpass);
n = length(iir_lowpass);
fft_iir_mag = abs(fft_iir);
fft_iir_mag = fft_iir_mag(1:floor(n/2)+1); %Only positive frequencies needed as fft gives mirror image.
fft_iir_mag(2:end-1) = 2 * fft_iir_mag(2:end-1); %Scaled amplitude for removing one half of the FFT.
frequencies_iir = (0:length(fft_iir_mag)-1) * fs /n; %Frequency vector.
figure;
plot(frequencies_iir/1000, fft_iir_mag); %/1000 for kHz

xlabel('Frequency (kHz)');
ylabel('Amplitude');
title('Frequency Domain: IIR Lowpass Filter Output');
grid on;
```

Figure 21: IIR lowpass filter design code

With the given information, a frequency response can be plotted using the freqz function. Since a and b transfer function are the same for my designed filter and the butter function, this can be plotted before implementing my design code, the butter function already gives the a and b values that will be used. 8192 is an arbitrary number chosen for the number of samples in the frequency response graph. It's high to give a high-resolution frequency response graph as seen in figure 22.

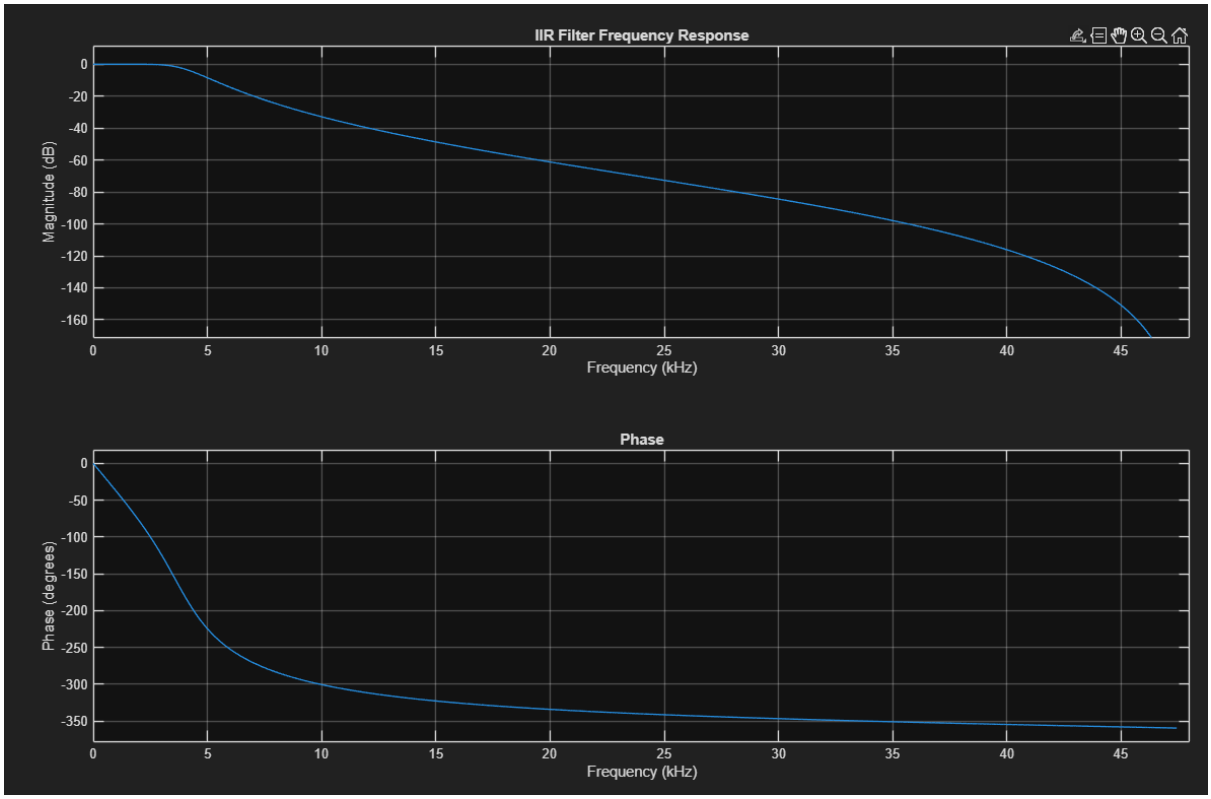

Figure 22: IIR Filter frequency response

Figure 22 shows that very low signals between 0-5kHz should have power decrease and then afterwards there is a curve downwards so that the higher the frequency the more the signals power decreases. This should make sure that the low frequency message should dominate the signal.

The structure of figure 20 is applied to each sample of the carrier recovery array as a for loop. Firstly, an IIR array is initialised as a 0 matrix the same size as the carrier recovery array as it will store the output of the filter. The code processes one sample of the carrier recovery at a time. The ff line will store the input samples and fb will store the past output components. Before every sample, the lines will shift for new values. There is an ongoing count which multiplies the samples with the respective weights and adds or subtracts depending on ff or fb.

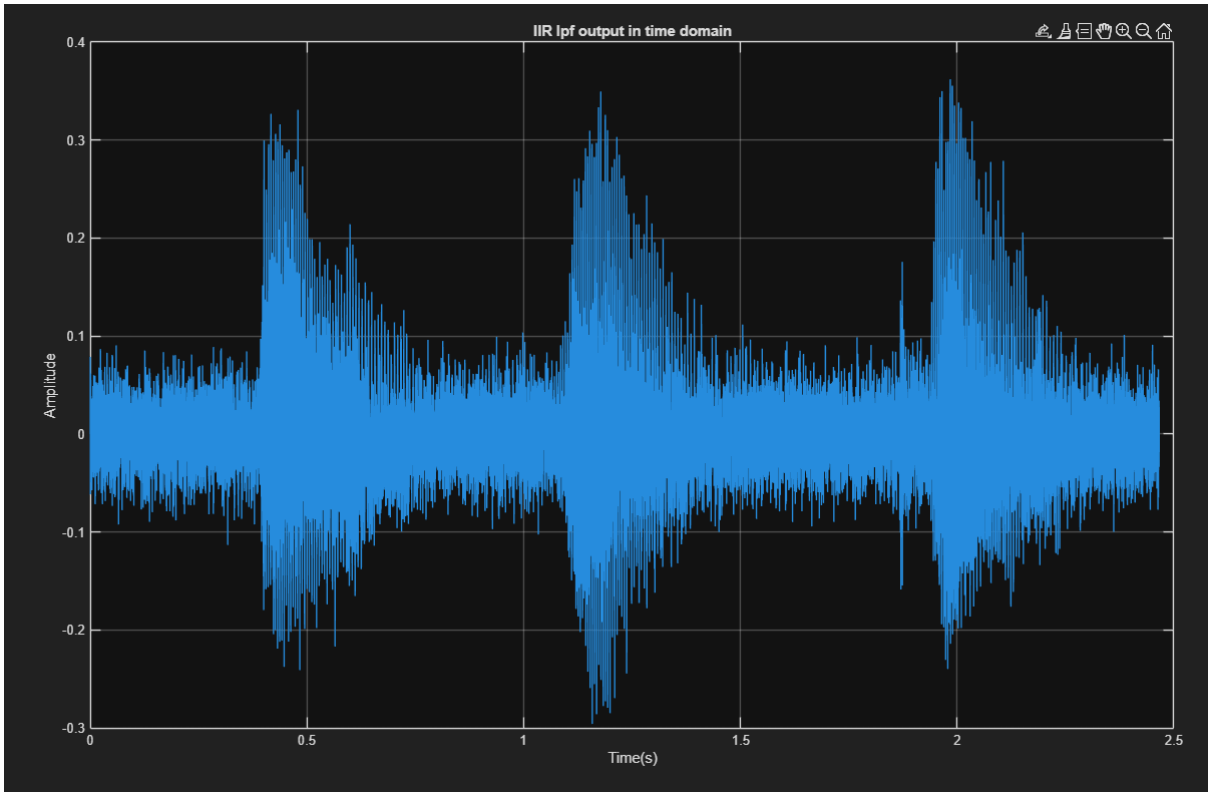

Figure 23: IIR lowpass filter output in the time domain (Message signal)

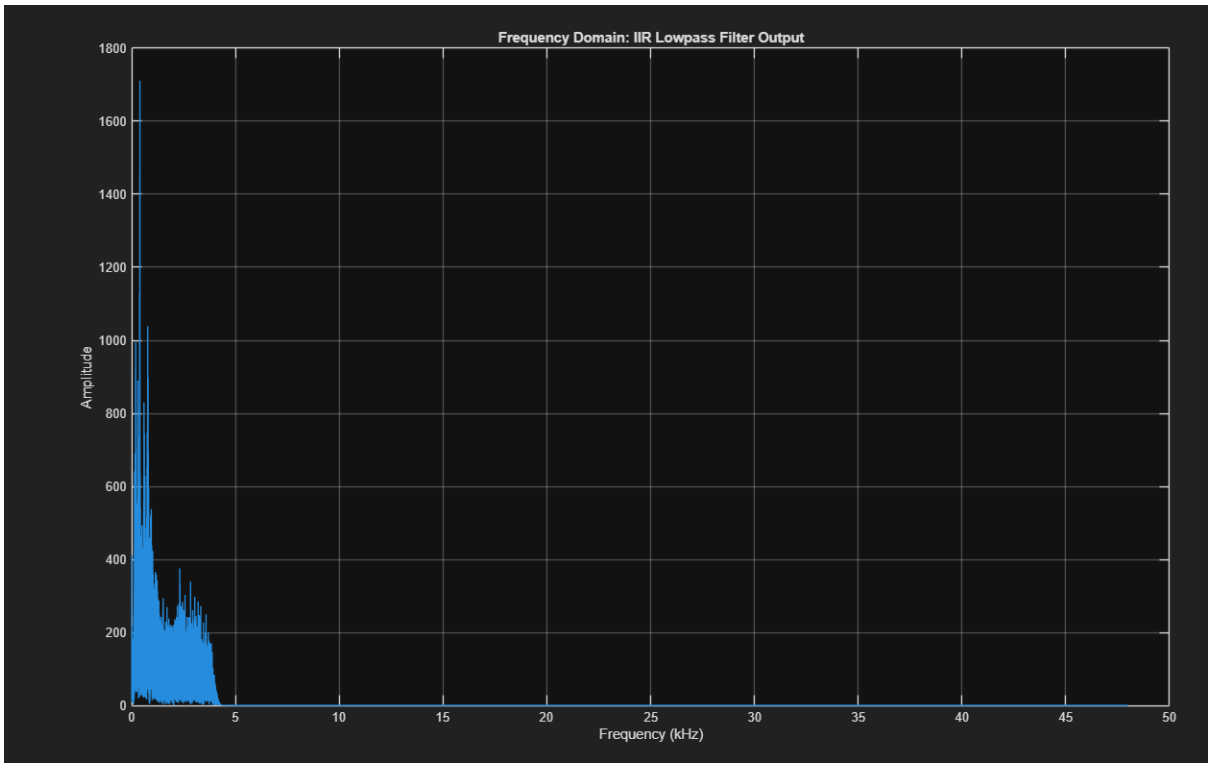

Figure 24: IIR lowpass filter output in frequency domain (Message signal)

Figure 24 shows that the 2*carrier frequency has now become negligent due to the IIR filter and we are left with just the message spectrum.

Task 5

```
%% Task 5, Audio signal output
fprintf('\nPlaying message...\n');
sound(iir_lowpass, fs);
```

Figure 25: Message output code

Changing phi and analysing the time domain, phi = 0 gave the highest amplitude in the time domain. The worst phi value was pi/2 as this gave the most noise and the message was not listenable.

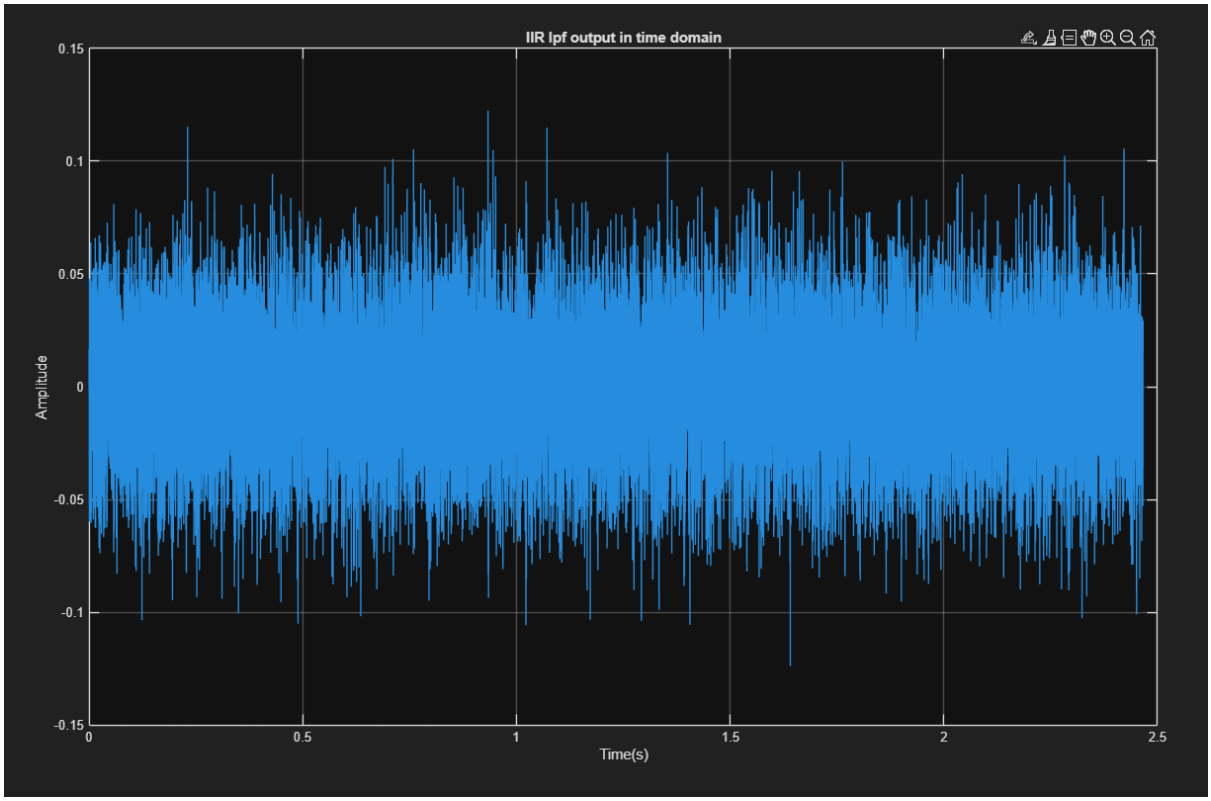

Figure 26: Worst phi value time signal

The message says “n, y, k”.